

Lab 3: Token Counter for Identifiers, Keywords & Operators Using Flex

Experiment Title

Token Counter for Identifiers, Keywords & Operators using Flex

Aim

To develop a lexical analyzer using Flex that scans input code and:

- Identifies tokens like keywords, identifiers, numbers, operators, and comments.
- Counts how many of each token type appears.

Software Requirements

Tool	Purpose
Flex	Lexical Analyzer Generator
GCC	C Compiler to compile generated code
OS	Windows (MinGW)

Theory

The purpose of a lexical analyzer is not only to recognize tokens but also to track their frequency for further compiler stages like optimization or symbol table generation.

In this experiment we:

- Use regular expressions to define token types (keywords, identifiers, operators, etc.).
- Maintain counters for each token type.
- Use Flex actions { ... } to print and count each token.

Program Structure

Flex File (token_counter.l)

```
%{  
#include <stdio.h>
```

```

int keywordCount    = 0;
int identifierCount = 0;
int numberCount     = 0;
int stringCount     = 0;
int logicalOpCount  = 0;
int operatorCount   = 0;
int delimiterCount  = 0;
int commentCount    = 0;
int unknownCount    = 0;
%}

%%

VV[^\n]*           { commentCount++; printf("COMMENT: %s\n", yytext); }
V\*([^\n]*|\/)*+V  { commentCount++; printf("COMMENT: %s\n", yytext); }
int|float|char|double|void|if|else|while|for|return|do|break|continue|switch|case|default
{ keywordCount++; printf("KEYWORD: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { identifierCount++; printf("IDENTIFIER: %s\n",
yytext); }
[0-9]+(\.[0-9]+)?   { numberCount++; printf("NUMBER: %s\n", yytext); }
\"([^\n]|\\.)*\n    { stringCount++; printf("STRING: %s\n", yytext); }
&&|\||\|           { logicalOpCount++; printf("LOGICAL_OP: %s\n", yytext); }
==|=|<|=|>|=|<|>|+|-|*|/|%=|?|= { operatorCount++; printf("OPERATOR: %s\n",
yytext); }
[{};,\[\]\(\)]      { delimiterCount++; printf("DELIMITER: %s\n", yytext); }
[ \t\n]+            ;
.                   { unknownCount++; printf("UNKNOWN: %s\n", yytext); }
%%

int main() {
    printf("Enter the code (Ctrl+Z then Enter to end input):\n");
    yylex();
    printf("\n-- Token Counts --\n");
    printf("Keywords    : %d\n", keywordCount);
    printf("Identifiers  : %d\n", identifierCount);
    printf("Numbers     : %d\n", numberCount);
    printf("Strings    : %d\n", stringCount);
    printf("Logical Ops : %d\n", logicalOpCount);
    printf("Operators  : %d\n", operatorCount);
    printf("Delimiters : %d\n", delimiterCount);
    printf("Comments  : %d\n", commentCount);
    printf("Unknown   : %d\n", unknownCount);
    return 0;
}

int yywrap() { return 1; }

```

Compilation and Execution Steps

```
flex token_counter.l  
gcc lex.yy.c -o token_counter  
./token_counter
```

Sample Input / Output

Input:

```
int x = 5;  
if (x > 2) x = x + 1;
```

Output (per token):

```
Keyword: int Identifier: x Operator: = Number: 5  
Delimiter: ; Keyword: if Delimiter: ( Identifier: x  
Operator: > Number: 2 Delimiter: ) Identifier: x  
Operator: = Identifier: x Operator: + Number: 1  
Delimiter: ;
```

Token Counts:

```
Keywords      : 2  
Identifiers   : 4  
Numbers       : 3  
Strings       : 0  
Logical Ops   : 0  
Operators     : 4  
Delimiters    : 5  
Comments      : 0  
Unknown       : 0
```

Result

- The lex program successfully detected and printed all relevant tokens.
- Counted token types such as keywords, operators, etc.
- Displayed the final count summary at the end of execution.

Conclusion

- Understood how to use regular expressions to detect various token types.
- Used Flex actions to maintain counters and print real-time results.